

IOT Air quality monitoring system

1 What each sensor does (function + "work" it does)

Sensor	Parameters you get (log separately)	What it <i>does</i> in the system (role)
MQ-135	"Air quality" / VOC / general toxic gas level (relative)	Detects "something is wrong in the air" – fumes, ammonia-ish, VOCs, bad air. Used as a general toxic/VOC index and to trigger early warnings.
MQ-7	CO level (relative), proxy for combustion/toxicity	Detects carbon monoxide from combustion (DG sets, boilers, forklifts). Acts as a toxicity + combustion issue indicator (poor burner, exhaust leakage).
MQ-2	Flammable gas level (relative)	Detects LPG / methane / hydrogen / flammable gas buildup . Used for explosion risk warning : "turn on ventilation / check for gas leak."
GP2Y1010AU0F	Dust concentration (relative fine particles)	Detects dust and fine particles (powder, aerosols) in air. Used to monitor cleanliness, dust exposure, filter performance .
DHT22	Temperature (°C)	Parameter 1: Ambient temperature – used to see if conditions are overheating or affecting processes/sensors.
DHT22	Humidity (%)	Parameter 2: Relative humidity – used to see if environment is too humid/dry, and for correcting/understanding gas sensor drift.

You'll model this in software as **6 signals**:

- `air_quality_index` (MQ-135 ADC)
- `co_level` (MQ-7 ADC)
- `flammable_level` (MQ-2 ADC)

- `dust_level` (GP2Y1010 ADC)
- `temperature_c` (DHT22)
- `humidity_pct` (DHT22)

2 ESP32 Pin Mapping (using Arduino core)

Assume a **generic ESP32 DevKit (DOIT)**.

We'll keep all analog signals on **ADC1 (GPIO32–39)** to avoid Wi-Fi ADC conflicts.

⚠ Important: MQ sensor modules are usually powered at 5V and their analog output can go up to 5V. You should use a simple resistor divider (e.g. 100k + 150k) to bring max voltage down to $\leq 3.3V$ before feeding ESP32 ADC pins.

🔌 Power rails

- **5V (VIN or 5V pin)** → MQ-135, MQ-7, MQ-2, GP2Y1010AU0F (Vcc)
- **3V3** → DHT22 Vcc
- **GND** → GND pins of *all* sensors

📋 Pin table: sensor → ESP32

🔧 1. MQ-135 – General air quality

MQ-135 Pin	Connects to	ESP32 Pin / Note
VCC	5V	ESP32 5V pin
GND	GND	Any ESP32 GND
AOUT	ADC input via divider	GPIO34 (ADC1_CH6) through 5V→3.3V divider
DOUT	<i>(optional)</i>	Can be ignored or wired to spare GPIO if you want digital threshold

🔧 2. MQ-7 – CO / toxicity

MQ-7 Pin	Connects to	ESP32 Pin / Note
VCC	5V	ESP32 5V pin
GND	GND	ESP32 GND
AOUT	ADC input via divider	GPIO35 (ADC1_CH7) via 5V→3.3V divider
DOUT	<i>(optional)</i>	Ignore or use as digital alarm input

3. MQ-2 – Flammable gas

MQ-2 Pin	Connects to	ESP32 Pin / Note
VCC	5V	ESP32 5V pin
GND	GND	ESP32 GND
AOUT	ADC input via divider	GPIO36 (ADC1_CH0) via 5V→3.3V divider
DOUT	<i>(optional)</i>	Ignore or hook to spare GPIO as digital alarm

4. GP2Y1010AU0F – Dust sensor

This sensor has **LED drive** and **analog output**.

Common pin names (check your PCB silkscreen/datasheet):

- **V-LED** – LED power
- **LED-GND** / **LED-** – LED ground
- **LED** / **LED_CTRL** – LED control pin
- **Vo** – analog output
- **VCC** – 5V
- **GND** – ground

Typical wiring:

GP2Y1010 Pin	Connects to	ESP32 Pin / Note
VCC	5V	ESP32 5V pin
GND	GND	ESP32 GND
LED (control)	ESP32 digital output via 150 Ω resistor	GPIO25 (drive LED high briefly to take reading)

GP2Y1010 Pin	Connects to	ESP32 Pin / Note
Vo	ADC input (0–3.6V)	GPIO32 (ADC1_CH4) – usually safe directly; keep under 3.3V if possible
LED-GND / V-LED other pins	According to module	Typically both return to GND or 5V via resistor; follow breakout's reference circuit

In code you'll:

1. Set `GPIO25` LOW most of the time.
2. To sample: set `GPIO25` HIGH for a short pulse, then read `analogRead(GPIO32)`.

5. DHT22 – Temperature + Humidity

DHT22 Pin	Connects to	ESP32 Pin / Note
VCC	3V3	ESP32 3V3 pin
GND	GND	ESP32 GND
DATA	Digital GPIO with pull-up	GPIO14 (any free digital pin is fine)
NC	Not connected	Ignore

Also add a **10k resistor between DATA and 3V3** (pull-up) close to the DHT22 just in case (might need for safety of the sensors)

In Arduino code you'll then read **two values** from this one sensor:

- `float temperatureC`
- `float humidityPct`

and treat them as two separate parameters.

Connections and pins for sensors summary goes like :

Sensor	Connection Type	ESP32 Pin
MQ-135	Analog (via divider)	GPIO34
MQ-7	Analog (via divider)	GPIO35
MQ-2	Analog (via divider)	GPIO36

Sensor	Connection Type	ESP32 Pin
GP2Y1010 (Vo)	Analog	GPIO32
GP2Y1010 (LED CTRL)	Digital	GPIO25
DHT22 (DATA)	Digital	GPIO14

All MQ sensors → **5V, GND, ADC pins**

DHT22 → **3.3V, GND, DATA**

Dust sensor → **5V, GND, GPIO32, GPIO25**

CODE FOR ARDUINO FOR THE ABOVE AND THE SAME

```
#include <DHT.h>

// ----- PIN DEFINITIONS -----

// Analog gas sensors (with 5V→3.3V dividers on AOUT)
#define MQ135_PIN  34  // Air quality / VOC
#define MQ7_PIN    35  // CO / toxicity
#define MQ2_PIN    36  // Flammable gas

// Dust sensor (GP2Y1010)
#define DUST_LED_PIN 25  // LED control (via ~150Ω resistor, active LOW)
#define DUST_ADC_PIN 32  // Vo output (analog)

// DHT22 temperature + humidity
#define DHTPIN      14
#define DHTTYPE     DHT22

// Update interval (ms)
const unsigned long SAMPLE_INTERVAL_MS = 1000; // 1000 ms = 1 sec
// const unsigned long SAMPLE_INTERVAL_MS = 2000; // ← use this for 2 sec updates

// ADC config
```

```

const uint8_t ADC_SAMPLES = 10; // average N samples per reading
const float  ADC_MAX      = 4095.0; // 12-bit ADC on ESP32

DHT dht(DHTPIN, DHTTYPE);

// ----- HELPER FUNCTIONS -----

// Average multiple ADC readings for stability
int readAnalogAveraged(uint8_t pin, uint8_t samples = ADC_SAMPLES) {
    long sum = 0;
    for (uint8_t i = 0; i < samples; i++) {
        sum += analogRead(pin);
        delay(2); // small delay between samples
    }
    return (int)(sum / samples);
}

// Read raw dust value from GP2Y1010
int readDustRaw() {
    // Pulse LED as per datasheet timing (approx)
    digitalWrite(DUST_LED_PIN, LOW); // LED ON (active LOW)
    delayMicroseconds(280); // wait before sampling
    int dustValue = analogRead(DUST_ADC_PIN);
    delayMicroseconds(40); // hold time
    digitalWrite(DUST_LED_PIN, HIGH); // LED OFF
    delayMicroseconds(9680); // wait until next cycle

    return dustValue;
}

// ----- SETUP -----

void setup() {
    Serial.begin(115200);
    delay(1000);
}

```

```

Serial.println();
Serial.println("ESP32 Air Quality Node – Booting...");

pinMode(DUST_LED_PIN, OUTPUT);
digitalWrite(DUST_LED_PIN, HIGH); // LED off by default (active LOW)

dht.begin();

// ESP32 default is 12-bit (0–4095), but we set explicitly
analogReadResolution(12);

Serial.println("Setup complete.");
}

// ----- LOOP -----

void loop() {
  unsigned long loopStart = millis();

  // ----- 1. Read analog gas sensors (0–4095 raw) -----
  int mq135Raw = readAnalogAveraged(MQ135_PIN);
  int mq7Raw   = readAnalogAveraged(MQ7_PIN);
  int mq2Raw   = readAnalogAveraged(MQ2_PIN);

  // ----- 2. Read dust sensor (GP2Y1010) -----
  int dustRaw = readDustRaw();

  // ----- 3. Read DHT22 (temperature + humidity) -----
  float temperatureC = dht.readTemperature();
  float humidityPct  = dht.readHumidity();
  bool dhtOk = !(isnan(temperatureC) || isnan(humidityPct));

  // ----- 4. OPTIONAL: Normalized 0–1 values (COMMENTED) -----
  // Uncomment these lines if you want normalized 0–1 values as well:
  //
  // float mq135Norm = mq135Raw / ADC_MAX;

```

```

// float mq7Norm  = mq7Raw  / ADC_MAX;
// float mq2Norm  = mq2Raw  / ADC_MAX;
// float dustNorm = dustRaw  / ADC_MAX;

// ----- 5. Print nicely to Serial -----
Serial.println("=====");
Serial.println("Raw ADC values (0-4095):");

Serial.printf("MQ-135 (air quality): %4d\n", mq135Raw);
Serial.printf("MQ-7  (CO level)  : %4d\n", mq7Raw);
Serial.printf("MQ-2  (flammable) : %4d\n", mq2Raw);
Serial.printf("Dust  (GP2Y1010)  : %4d\n", dustRaw);

if (dhtOk) {
    Serial.printf("Temperature (C): %.2f\n", temperatureC);
    Serial.printf("Humidity   (%): %.2f\n", humidityPct);
} else {
    Serial.println("DHT22 read failed (NaN) – check wiring / power.");
}

// ----- 6. OPTIONAL: Print normalized values (COMMENTED) -----
// Uncomment this whole block if you also want to see 0-1 normalized:
//
// Serial.println("\nNormalized values (0-1):");
// Serial.printf("MQ-135 norm: %.3f\n", mq135Norm);
// Serial.printf("MQ-7  norm: %.3f\n", mq7Norm);
// Serial.printf("MQ-2  norm: %.3f\n", mq2Norm);
// Serial.printf("Dust  norm: %.3f\n", dustNorm);

// ----- 7. JSON-style payload (easy for parsing/logging) -----
Serial.println("\nAs JSON-like payload:");
Serial.print("{");
Serial.printf("\"mq135_raw\":%d,", mq135Raw);
Serial.printf("\"mq7_raw\":%d,",  mq7Raw);
Serial.printf("\"mq2_raw\":%d,",  mq2Raw);
Serial.printf("\"dust_raw\":%d,", dustRaw);

```



```

if (dhtOk) {
  Serial.printf("\ntemperature_c\":%.2f,", temperatureC);
  Serial.printf("\nhumidity_pct\":%.2f", humidityPct);
} else {
  Serial.print("\ntemperature_c\":null,\nhumidity_pct\":null");
}
Serial.println("{}");
Serial.println();

// ----- 8. Maintain ~1s or 2s update interval -----
unsigned long elapsed = millis() - loopStart;
if (elapsed < SAMPLE_INTERVAL_MS) {
  delay(SAMPLE_INTERVAL_MS - elapsed);
}
}

```

PHASE 2 :

1 Firebase setup (one-time, super simple)

1. Go to **Firestore Console** → **Your Project**.
2. **Build** → **Realtime Database** → **Create database**.
3. Choose region, then set **rules** to test/dev mode:

```

{
  "rules": {
    ".read": true,
    ".write": true}
}

```

⚠ This means no authentication, fully open. Good for lab/prototype, not for production.

4. From the Database page, copy your DB URL; it'll look like:

- `https://your-project-id-default-rtdb.firebaseio.com`

or

- `https://your-project-id-region.firebaseiodatabase.app`

We'll call this `FIREBASE_HOST`.

2 Arduino libraries you need

In **Arduino IDE** (ESP32 board already installed):

1. DHT sensor libraries

- `DHT sensor library` by *Adafruit*
- `Adafruit Unified Sensor` (dependency)

Install via:

Sketch → Include Library → Manage Libraries...

1. No extra Firebase library needed

We'll use:

- `WiFi.h`
- `HTTPClient.h`
- `WiFiClientSecure.h`

These come with the ESP32 core.

3 Full ESP32 + Firebase code (REST POST every 2 seconds)

This builds on your sensor setup:

- MQ-135 → GPIO34
- MQ-7 → GPIO35
- MQ-2 → GPIO36

- GP2Y1010: LED → GPIO25, Vo → GPIO32
- DHT22 → GPIO14

👉 Replace:

```
#define WIFI_SSID    "YOUR_WIFI_SSID"
#define WIFI_PASSWORD "YOUR_WIFI_PASSWORD"
#define FIREBASE_HOST "https://your-project-id-default-rtdb.firebaseio.co
m"
```

with your actual values.

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <DHT.h>

// ----- WIFI + FIREBASE CONFIG -----

#define WIFI_SSID    "YOUR_WIFI_SSID"
#define WIFI_PASSWORD "YOUR_WIFI_PASSWORD"

// From Firebase Realtime Database URL (no trailing slash)
#define FIREBASE_HOST "https://your-project-id-default-rtdb.firebaseio.co
m"

// Device identifier (for DB path)
String deviceId = "esp32_node_01";

// ----- SENSOR PINS -----

// Analog gas sensors (with 5V→3.3V divider on AOUT)
#define MQ135_PIN    34  // Air quality / VOC
#define MQ7_PIN      35  // CO / toxicity
#define MQ2_PIN      36  // Flammable gas
```

```

// Dust sensor (GP2Y1010)
#define DUST_LED_PIN 25 // LED control (via ~150Ω resistor, active LOW)
#define DUST_ADC_PIN 32 // Vo output (analog)

// DHT22 temperature + humidity
#define DHTPIN 14
#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

// Timing config
const unsigned long SAMPLE_INTERVAL_MS = 2000; // 2 seconds
const uint8_t ADC_SAMPLES = 10;
const float ADC_MAX = 4095.0;

// ----- HELPER FUNCTIONS -----

// Average multiple ADC readings for stability
int readAnalogAveraged(uint8_t pin, uint8_t samples = ADC_SAMPLES) {
    long sum = 0;
    for (uint8_t i = 0; i < samples; i++) {
        sum += analogRead(pin);
        delay(2);
    }
    return (int)(sum / samples);
}

// Read dust raw value from GP2Y1010
int readDustRaw() {
    digitalWrite(DUST_LED_PIN, LOW); // LED ON (active LOW)
    delayMicroseconds(280);
    int dustValue = analogRead(DUST_ADC_PIN);
    delayMicroseconds(40);
    digitalWrite(DUST_LED_PIN, HIGH); // LED OFF
    delayMicroseconds(9680);
    return dustValue;
}

```

```

}

void connectWiFi() {
  Serial.printf("Connecting to WiFi: %s\n", WIFI_SSID);
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  int retries = 0;
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
    retries++;
    if (retries > 40) {
      Serial.println("\nWiFi failed, restarting...");
      ESP.restart();
    }
  }
  Serial.println("\nWiFi connected.");
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());
}

// ----- SETUP -----

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println();
  Serial.println("ESP32 Air Quality Node → Firebase (no auth)");

  pinMode(DUST_LED_PIN, OUTPUT);
  digitalWrite(DUST_LED_PIN, HIGH); // LED off by default (active LOW)

  dht.begin();
  analogReadResolution(12); // 0-4095

```

```

    connectWiFi();
}

// ----- LOOP -----

void loop() {
    unsigned long startMs = millis();

    // 1. Read sensors
    int mq135Raw = readAnalogAveraged(MQ135_PIN);
    int mq7Raw  = readAnalogAveraged(MQ7_PIN);
    int mq2Raw  = readAnalogAveraged(MQ2_PIN);
    int dustRaw = readDustRaw();

    float temperatureC = dht.readTemperature();
    float humidityPct  = dht.readHumidity();
    bool dhtOk = !(isnan(temperatureC) || isnan(humidityPct));

    // ----- OPTIONAL NORMALIZATION (0-1) -----
    // Uncomment if you want normalized values:
    //
    // float mq135Norm = mq135Raw / ADC_MAX;
    // float mq7Norm   = mq7Raw   / ADC_MAX;
    // float mq2Norm   = mq2Raw   / ADC_MAX;
    // float dustNorm  = dustRaw  / ADC_MAX;

    // 2. Debug print
    Serial.println("----- Sensor Readings -----");
    Serial.printf("MQ-135: %d\n", mq135Raw);
    Serial.printf("MQ-7  : %d\n", mq7Raw);
    Serial.printf("MQ-2  : %d\n", mq2Raw);
    Serial.printf("Dust  : %d\n", dustRaw);

    if (dhtOk) {
        Serial.printf("Temp (C): %.2f\n", temperatureC);
        Serial.printf("Hum  (%): %.2f\n", humidityPct);
    }
}

```

```

    } else {
        Serial.println("DHT22 error (NaN).");
    }

    // 3. Build JSON payload (plain String)
    String json = "{";
    json += "\"mq135_raw\":" + String(mq135Raw) + ",";
    json += "\"mq7_raw\":" + String(mq7Raw) + ",";
    json += "\"mq2_raw\":" + String(mq2Raw) + ",";
    json += "\"dust_raw\":" + String(dustRaw) + ",";

    if (dhtOk) {
        json += "\"temperature_c\":" + String(temperatureC, 2) + ",";
        json += "\"humidity_pct\":" + String(humidityPct, 2) + ",";
    } else {
        json += "\"temperature_c\":null,";
        json += "\"humidity_pct\":null,";
    }
}

// OPTIONAL normalized values – uncomment if used above:
// json += "\"mq135_norm\":" + String(mq135Norm, 3) + ",";
// json += "\"mq7_norm\":" + String(mq7Norm, 3) + ",";
// json += "\"mq2_norm\":" + String(mq2Norm, 3) + ",";
// json += "\"dust_norm\":" + String(dustNorm, 3) + ",";

// simple device-side timestamp
json += "\"ts_ms\":" + String(millis());
json += "}";

Serial.println("JSON payload:");
Serial.println(json);

// 4. Send to Firebase via REST POST (append new child)
if (WiFi.status() == WL_CONNECTED) {
    WiFiClientSecure client;
    client.setInsecure(); // ⚠ ignore SSL cert validation (ok for dev/prototype)

```

```

HTTPClient https;

// Path: /nodes/<deviceId>/readings.json
String url = String(FIREBASE_HOST) + "/nodes/" + deviceId + "/readings.js
on";

Serial.print("POST to: ");
Serial.println(url);

if (https.begin(client, url)) {
  https.addHeader("Content-Type", "application/json");

  int httpCode = https.POST(json);
  if (httpCode > 0) {
    Serial.printf("HTTP POST code: %d\n", httpCode);
    String payload = https.getString();
    Serial.println("Firebase response:");
    Serial.println(payload); // contains {"name":"-NxAbCdEf..."} (auto key)
  } else {
    Serial.printf("HTTP POST failed, error: %s\n", https.errorToString(httpCod
e).c_str());
  }

  https.end();
} else {
  Serial.println("HTTPS begin() failed.");
}
} else {
  Serial.println("WiFi not connected, skipping Firebase POST.");
  connectWiFi();
}

// 5. Maintain ~2 second interval
unsigned long elapsed = millis() - startMs;
if (elapsed < SAMPLE_INTERVAL_MS) {

```



```
    delay(SAMPLE_INTERVAL_MS - elapsed);  
  }  
}
```

4 What happens in Firebase

With this code, every ~2 seconds you get a new child under:

`/nodes/esp32_node_01/readings/<auto_generated_key>`

Each node looks like:

```
{  
  "mq135_raw": 1234,  
  "mq7_raw": 567,  
  "mq2_raw": 890,  
  "dust_raw": 2345,  
  "temperature_c": 28.75,  
  "humidity_pct": 61.20,  
  "ts_ms": 12345678  
}
```

So your dashboard later can:

- Subscribe to `/nodes/esp32_node_01/readings`
- Stream changes in real-time
- Draw charts / gauges per parameter

THE libraries for audrino might change cause the Amrita boards are somewhat different at times and you get a different one sometimes, so borrow it from the college itself dont buy a esp32

the code isnt perfect so tweak it according to the project and the board.

- make a few realtime charts using the data and thresholds to simulate alerts to give it a better touch.